

Arquitectura de Software

Taller 1 / Presentación 1 — Grupo 3 (G3)

Documento Técnico

DecisionRoom G3

Sistema de Votación y Toma de Decisiones en Tiempo Real

Parte 1: Investigación del estilo y del stack tecnológico

Estilo arquitectónico: Clean Architecture

Frontend: Electron + Python (Eel)

Backend: Ruby on Rails (modo API)

Persistencia: Redis

Protocolo de integración: WebSockets

Contenerización: Docker / Docker Compose

Repositorio: <https://github.com/KatheBravo/taller1-arqui-G3>

Integrantes: Daniel Avila, Katheryn Bravo, Juan Sebastian Alvarez

Índice

1. Contexto del sistema (punto de partida)	5
1.1. Propósito y alcance	5
1.2. Stack tecnológico asignado	5
1.3. Estructura del repositorio	5
1.4. Modelo de datos y su representación en Redis	6
1.5. Método y limitaciones de este documento	6
2. Investigación del estilo y stack	6
2.1. Estilo arquitectónico: Clean Architecture	6
2.1.1. Definición clara (qué es y qué no es)	6
2.1.2. Clasificación del estilo	7
2.1.3. Características principales	7
2.1.4. Historia y evolución	8
2.1.5. Ventajas y desventajas	8
2.1.6. Problemas comunes que se presentan y patrones	9
2.1.7. Patrones aplicables y cuándo usarlos	9
2.1.8. Casos de uso (cuándo usarlo y cuándo no)	10
2.1.9. Casos de aplicación (ejemplos reales en la industria)	10
2.2. Frontend: Electron + Python (Eel)	11
2.2.1. Definición clara (qué es y qué no es)	11
2.2.2. Características principales	11
2.2.3. Historia y evolución	12
2.2.4. Ventajas y desventajas	12
2.2.5. Casos de uso (cuándo usarlo y cuándo no)	12
2.2.6. Casos de aplicación (ejemplos reales en la industria)	13
2.3. Backend: Ruby on Rails (modo API)	13
2.3.1. Definición clara (qué es y qué no es)	13
2.3.2. Características principales	13
2.3.3. Historia y evolución	13
2.3.4. Ventajas y desventajas	14
2.3.5. Casos de uso (cuándo usarlo y cuándo no)	14
2.3.6. Casos de aplicación (ejemplos reales en la industria)	15
2.4. Persistencia: Redis	15
2.4.1. Definición clara (qué es y qué no es)	15
2.4.2. Características principales	15

2.4.3.	Historia y evolución	15
2.4.4.	Ventajas y desventajas	16
2.4.5.	Casos de uso (cuándo usarlo y cuándo no)	16
2.4.6.	Casos de aplicación (ejemplos reales en la industria)	16
2.5.	Protocolo de integración: WebSockets	17
2.5.1.	Definición clara (qué es y qué no es)	17
2.5.2.	Características principales	17
2.5.3.	Historia y evolución	17
2.5.4.	Ventajas y desventajas	17
2.5.5.	Casos de uso (cuándo usarlo y cuándo no)	18
2.5.6.	Casos de aplicación (ejemplos reales en la industria)	18
2.6.	Contenerización: Docker y Docker Compose	18
2.6.1.	Definición clara (qué es y qué no es)	18
2.6.2.	Características principales	18
2.6.3.	Historia y evolución	18
2.6.4.	Ventajas y desventajas	19
2.6.5.	Casos de uso (cuándo usarlo y cuándo no)	19
2.6.6.	Casos de aplicación (ejemplos reales en la industria)	19
2.7.	Relación entre el estilo y las tecnologías seleccionadas	19
2.7.1.	Cómo se materializa el estilo en el repositorio	19
2.7.2.	Por qué encajan	20
2.7.3.	Tensiones detectadas entre el estilo y el stack	20
2.8.	Qué tan común es el stack designado	22
2.8.1.	Prevalencia individual de cada tecnología	22
2.8.2.	Relación entre las tecnologías	22
2.8.3.	Conclusión	23
3.	Fase de diseño: matrices de análisis arquitectónico	23
3.1.	Matriz 1: Atributos de calidad vs Estilo arquitectónico	23
3.2.	Matriz 2: Principios de diseño vs Estilo arquitectónico	24
3.3.	Matriz 3: Tácticas arquitectónicas vs Estilo y stack	26
3.4.	Matriz 4: Mercado laboral vs Estilo y stack	28
4.	Diagramas de arquitectura	31
4.1.	Diagrama de contexto (C4 — Nivel 1)	31
4.2.	Diagrama de contenedores (C4 — Nivel 2)	32
4.3.	Diagrama de alto nivel (arquitectura general)	33

4.4. Diagrama de secuencia: registrar un voto	33
4.5. Diagrama entidad–relación	34
4.6. Diagrama de despliegue	35
5. Lecciones aprendidas	35

Índice de figuras

1.	Diagrama de contexto del sistema de votación DecisionRoom, en modo kiosco . . .	31
2.	Diagrama de contenedores: aplicación de kiosco, backend Rails y almacén Redis . .	32
3.	Diagrama de alto nivel del sistema de votación: arquitectura general y flujo principal de comunicación	33
4.	Diagrama de secuencia del caso de uso RegisterVote , de la selección del voto a la actualización de la pantalla	33
5.	Modelo entidad-relación de las votaciones, sus opciones y los votos registrados . .	34
6.	Diagrama de despliegue del entorno local unificado	35

Índice de tablas

28.	Atributos de calidad (ISO/IEC 25010:2023) frente al estilo Clean Architecture . . .	23
29.	Principios de diseño de software frente al estilo Clean Architecture	25
30.	Tácticas arquitectónicas frente al estilo Clean Architecture y al stack tecnológico .	27
31.	Mercado laboral: demanda y remuneración por tecnología del stack	29

1. Contexto del sistema (punto de partida)

1.1. Propósito y alcance

DecisionRoom G3 es una plataforma distribuida pensada para salas de juntas, asambleas de socios y comités técnicos. Permite que varios delegados, conectados desde sus estaciones de trabajo, emitan votos sobre mociones y vean actualizarse los resultados en tiempo real a través de un canal WebSocket bidireccional [1].

El repositorio implementa un caso de uso principal (registrar un voto) y uno de consulta (obtener los resultados). Las reglas de negocio implementadas son: la votación debe existir, debe estar abierta, la opción debe pertenecer a la votación y cada terminal solo puede votar una vez [1].

1.2. Stack tecnológico asignado

Capa	Tecnología	Rol en el sistema
Estilo arquitectónico	Clean Architecture	Organización interna del backend
Frontend	Electron + Python (Eel)	Terminal de escritorio del delegado
Backend	Ruby on Rails (modo API, sin ActiveRecord)	Casos de uso y canal WebSocket
Persistencia	Redis	Conteos, votantes y registro de votos
Protocolo de integración	WebSockets (ActionCable)	Envío de votos y difusión de resultados
Contenerización	Docker / Docker Compose	Backend y Redis

1.3. Estructura del repositorio

Ruta	Contenido
backend/app/core/entities/	Entidades <code>Poll</code> , <code>Option</code> y <code>Vote</code> (clases Ruby simples)
backend/app/core/use_cases/	<code>RegisterVote</code> y <code>GetPollResults</code>
backend/app/core/errors.rb	Jerarquía de errores de dominio
backend/app/infrastructure/repositories/	<code>RedisVoteRepository</code>
backend/app/channels/	<code>VotingChannel</code> (adaptador WebSocket)
backend/app/controllers/	<code>PollsController</code> (alternativa HTTP)
backend/config/	Configuración de Rails, ActionCable, Redis y Puma
frontend_desktop/	Cliente Eel: <code>main.py</code> y carpeta <code>web/</code>
tests/	Prueba de extremo a extremo y script de endpoints
docker-compose.yml	Servicios <code>redis</code> y <code>backend</code>
docs/	Documentación técnica y de código

1.4. Modelo de datos y su representación en Redis

El dominio define tres entidades: **Poll** (votación), **Option** (opción) y **Vote** (voto). Su persistencia en Redis se organiza en las siguientes claves [1]:

Clave	Tipo	Uso
poll:{id}	Hash	Metadatos de la votación
poll:{id}:options	Hash	Contador de votos por opción
poll:{id}:option_details	Hash	Texto de cada opción
poll:{id}:voters	Set	Identificadores de quienes ya votaron
vote:{uuid}	Hash	Registro individual de auditoría

1.5. Método y limitaciones de este documento

Las afirmaciones sobre el repositorio se verificaron contra el código del commit `2adfe51`. Las afirmaciones sobre tecnologías se sustentan en las fuentes listadas al final. Donde no se encontró evidencia pública verificable, se indica de forma expresa.

Limitaciones declaradas

- Las fechas de las publicaciones originales de Palermo (Onion, 2008) y Martin (Clean Architecture, 2012; libro, 2017) provienen de la literatura secundaria consultada y deben contrastarse con las publicaciones originales antes de la entrega.
- El caso de Nubank se documenta a partir de una copia pública de un repositorio de ejemplo que se le atribuye, no de una publicación corporativa.
- Para Eel no se encontraron casos de aplicación industrial documentados.

2. Investigación del estilo y stack

2.1. Estilo arquitectónico: Clean Architecture

2.1.1. Definición clara (qué es y qué no es)

Qué es. Clean Architecture es un estilo arquitectónico propuesto por Robert C. Martin que organiza un sistema en círculos concéntricos, cada uno con una responsabilidad. Su regla central es la *Regla de Dependencia*: las dependencias del código fuente solo pueden apuntar hacia adentro, es decir, hacia las políticas de más alto nivel [2]. El diagrama canónico contiene cuatro anillos: entidades, casos de uso, adaptadores de interfaz, y frameworks y controladores externos.

Microsoft describe este estilo como aquel al que tienden las aplicaciones que aplican el Principio de Inversión de Dependencias junto con Domain-Driven Design, y lo sitúa en el centro de la aplicación como *lógica de negocio y modelo de aplicación* [6].

Qué no es.

- **No es una biblioteca ni un framework:** es una regla de organización y de dependencias.
- **No es una plantilla fija de carpetas:** los nombres `core/` o `infrastructure/` son una elección del equipo; lo esencial es la dirección de las dependencias.

- **No es un patrón de diseño de clase** como Singleton u Observer; actúa sobre todo el sistema.
- **No exige exactamente cuatro círculos:** el número puede variar según la complejidad de la aplicación [12].
- **No es una arquitectura por capas tradicional.** En esta última las dependencias de compilación van de arriba hacia abajo (interfaz $\rightarrow$ lógica $\rightarrow$ datos), por lo que la lógica de negocio depende de los detalles de acceso a datos y resulta difícil de probar sin una base de datos [6].
- **No es un modelo de despliegue:** describe la organización interna del código y es compatible con un despliegue monolítico [6].

2.1.2. Clasificación del estilo

Se propone clasificarlo como un **estilo estructural de vista de módulos**: describe unidades de código y sus relaciones de dependencia, no componentes en ejecución y sus conectores. Dentro de esa vista pertenece a la familia de las arquitecturas **centradas en el dominio**, junto con la Arquitectura Hexagonal y la Arquitectura Cebolla (*Onion*) [6, 5].

Estilo	Autor y año	Énfasis
Hexagonal (puertos y adaptadores)	A. Cockburn, 2005	Componentes de aplicación conectables a su entorno mediante puertos y adaptadores; permite intercambiar componentes y facilita la automatización de pruebas [5]
Onion	J. Palermo, 2008	Modelo de dominio en el centro y dependencias hacia él [4]
Clean Architecture	R. C. Martin, 2012	Síntesis de las anteriores en cuatro anillos y Regla de Dependencia explícita [2]

Los tres comparten el mismo principio de fondo: el negocio ocupa el centro y las preocupaciones de infraestructura quedan en la periferia [6].

2.1.3. Características principales

- **Regla de Dependencia.** Los círculos internos no conocen a los externos. Nada declarado en un círculo externo (nombres, clases, funciones) se menciona en uno interno [2].
- **Independencia de frameworks.** El framework se trata como una herramienta, no como la estructura del sistema.
- **Independencia de la interfaz de usuario.** La interfaz puede cambiar sin alterar las reglas de negocio.
- **Independencia de la base de datos.** Las reglas de negocio no conocen el motor de persistencia.
- **Testabilidad.** Las reglas de negocio se prueban sin interfaz, base de datos ni servidor web [7].
- **Inversión de Dependencias en las fronteras.** Cuando un caso de uso necesita algo del exterior (por ejemplo, guardar datos), declara una abstracción propia y la infraestructura la

implementa. Así, en tiempo de compilación la infraestructura depende del núcleo, aunque en tiempo de ejecución el núcleo la invoque [7].

- **Datos simples en las fronteras.** Lo que cruza un límite entre círculos son estructuras de datos sencillas, no objetos del framework ni filas de base de datos [2].

2.1.4. Historia y evolución

Periodo	Hito
Décadas previas	Predominio de la arquitectura por capas con dependencias de arriba hacia abajo, que acopla la lógica de negocio al acceso a datos [6]
2005	Cockburn propone la arquitectura hexagonal (puertos y adaptadores) [5]
2008	Palermo formula la arquitectura Onion [4]
2012	Martin publica <i>The Clean Architecture</i> y unifica estas ideas [2]
2017	Martin publica el libro <i>Clean Architecture</i> [3]. Ese mismo año Microsoft adopta el nombre Clean Architecture como base de su guía y aplicación de referencia <i>eShopOnWeb</i> para ASP.NET Core [8]
2024	Microsoft (equipo ISE) documenta una plantilla base con Clean Architecture para .NET, orientada a aplicaciones medianas y grandes [9]

La evolución ha sido de convergencia: las guías de Microsoft señalan que este enfoque ha recibido nombres sucesivos (hexagonal, puertos y adaptadores, Onion y Clean) y adoptan el último como referencia [6].

2.1.5. Ventajas y desventajas

Aspecto	Tipo	Descripción
Mantenibilidad	Ventaja	Los cambios en interfaz o persistencia quedan localizados en los círculos externos [7]
Testabilidad	Ventaja	La lógica de negocio se prueba sin base de datos, evitando la dependencia que introduce la arquitectura por capas tradicional [6]
Reemplazabilidad	Ventaja	Los detalles de implementación dependen de abstracciones de nivel superior, lo que vuelve las aplicaciones más modulares y mantenibles [7]
Trabajo en equipo	Ventaja	Cada persona puede trabajar sobre su capa con menor riesgo de conflictos [12]
Longevidad	Ventaja	Reduce el costo de migrar o actualizar dependencias obsoletas [12]
Complejidad inicial	Desventaja	Requiere interfaces, adaptadores y conversión de datos en las fronteras, que aumentan la cantidad de código
Curva de aprendizaje	Desventaja	Exige comprender la inversión de dependencias y la disciplina de capas

Aspecto	Tipo	Descripción
Sobrecosto en proyectos pequeños	Desventaja	En aplicaciones sencillas puede resultar redundante y no aprovechar todo su potencial [12]
Aplicación superficial	Desventaja	Es posible tener las carpetas del estilo sin cumplir la regla de dependencia, con lo que se pierde el beneficio

2.1.6. Problemas comunes que se presentan y patrones

Problema	Descripción	Solución o patrón
Fuga de infraestructura al dominio	Tipos de la base de datos o parámetros HTTP llegan a los casos de uso	Objetos de transferencia de datos (DTO) y objetos propios del dominio en las fronteras; abstracciones que no revelen su implementación [10]
Acoplamiento en la instanciación	El caso de uso crea directamente la clase concreta de persistencia	Inversión de Dependencias e Inyección de Dependencias: el colaborador se recibe desde afuera [7]
Lógica de negocio en controladores	Reglas que deberían estar en los casos de uso quedan en la capa de entrega	Mover las reglas a casos de uso; el controlador solo traduce y delega
Errores de infraestructura visibles	Excepciones propias del motor de datos llegan al núcleo	Traducirlas a errores de dominio en el adaptador
Exceso de conversiones	Mapeos innecesarios entre modelos casi idénticos	Limitar las conversiones a fronteras donde el cambio de modelo aporta valor
Dependencia del framework en el núcleo	Entidades atadas a un ORM o a clases del framework	Entidades como objetos simples y repositorios como frontera [10]

2.1.7. Patrones aplicables y cuándo usarlos

Patrón	Propósito	Cuándo usarlo	Aplicación en DecisionRoom
Repositorio	Encapsular el acceso a datos con un lenguaje del negocio	Cuando el núcleo necesita persistir o consultar datos	<code>RedisVoteRepository</code>
Caso de uso (Interactor)	Un objeto por acción del usuario, con un único punto de entrada	Cuando hay reglas de aplicación que deben probarse de forma aislada	<code>RegisterVote</code> , <code>GetPollResults</code>
Adaptador	Traducir entre el formato externo y el interno	En cada frontera de entrada o salida	<code>VotingChannel</code> , <code>PollsController</code>

Patrón	Propósito	Cuándo usarlo	Aplicación en DecisionRoom
DTO	Transportar datos entre círculos sin exponer el modelo interno	Cuando el formato externo difiere del del dominio	Payload de eventos (results_updated)
Inyección de Dependencias	Entregar colaboradores desde fuera	Siempre que un caso de uso necesite infraestructura	Parámetros repository: y broadcaster:
Publicación/suscripción	Difundir eventos a varios receptores desacoplados	Cuando varios clientes deben enterarse de un cambio	Difusión de resultados mediante ActionCable

2.1.8. Casos de uso (cuándo usarlo y cuándo no)

Usarlo cuando...	Evitarlo cuando...
El negocio tiene reglas complejas o significativas	La aplicación es un CRUD simple
Se esperan varios canales de entrada (web, móvil, escritorio, eventos)	Es un prototipo o una prueba de concepto desechable
El sistema vivirá muchos años y se mantendrá con equipos cambiantes	El valor está en el framework y no en la lógica propia
La tecnología externa cambiará con el tiempo	El equipo es muy pequeño y el plazo muy corto
Se necesita alta capacidad de prueba de la lógica de negocio	No hay reglas de negocio que proteger

Microsoft lo recomienda especialmente para aplicaciones medianas y grandes con lógica de negocio significativa y necesidades de prueba altas [9]; la literatura divulgativa advierte que en proyectos pequeños puede ser redundante [12].

2.1.9. Casos de aplicación (ejemplos reales en la industria)

Caso	Tipo de evidencia	Descripción
Microsoft — eShopOnWeb	Aplicación de referencia oficial	Organiza su código en proyectos siguiendo Clean Architecture; se ejecuta en contenedores con Docker Compose [6]
Plantilla <i>Clean Architecture</i> para ASP.NET Core	Plantilla comunitaria citada por Microsoft	Punto de partida para soluciones con Clean Architecture; Microsoft la referencia en su guía [6, 10]

Caso	Tipo de evidencia	Descripción
Nubank	Repositorio de ejemplo atribuido	Organiza sus microservicios con arquitectura de puertos y adaptadores (hexagonal), en capas de lógica, controladores, adaptadores y puertos [11]
Microsoft ISE	Repositorio base publicado	Plantilla .NET con Clean Architecture para aplicaciones con lógica de negocio significativa [9]

Alcance de la evidencia

La arquitectura interna de un sistema comercial rara vez se declara públicamente con el nombre exacto del estilo. Por eso los casos anteriores combinan guías de referencia oficiales y estilos afines. No se incluyen empresas para las que no se halló evidencia pública verificable.

2.2. Frontend: Electron + Python (Eel)

Precisión sobre el repositorio

En el repositorio, `frontend_desktop/main.py` inicia Eel con los modos `chrome`, luego `edge` y por último `default`, y `requirements.txt` no incluye Electron. Eel admite un modo `electron` [16], pero la implementación actual abre la interfaz en un navegador Chromium en modo aplicación. Este punto debe declararse en la sustentación.

2.2.1. Definición clara (qué es y qué no es)

Electron es un framework de código abierto para construir aplicaciones de escritorio multiplataforma con HTML, CSS y JavaScript. Integra el motor de renderizado Chromium y el entorno de ejecución Node.js, y estructura la aplicación en un proceso principal y varios procesos de renderizado que se comunican por un módulo de comunicación entre procesos [14, 15]. *No es* un navegador web, ni un framework de interfaz como React, ni un compilador a código nativo: cada aplicación incluye su propio runtime.

Eel es una pequeña biblioteca de Python para crear aplicaciones de interfaz gráfica “tipo Electron”, sin conexión, con HTML y JavaScript. Levanta un servidor web local y permite anotar funciones de Python para llamarlas desde JavaScript y viceversa [16]. *No es* un motor de renderizado (usa un navegador ya instalado), ni un empaquetador (el empaquetado se hace con herramientas como PyInstaller), ni una solución pensada para aplicaciones de gran tamaño: se define a sí misma como orientada a aplicaciones de interfaz *cortas y simples* [16].

2.2.2. Características principales

Tecnología	Características
Electron	Motor Chromium y Node.js integrados; arquitectura de procesos principal y de renderizado; distribución para Windows, macOS y Linux; ecosistema npm; licencia MIT en sus componentes originales [14]

Tecnología	Características
Eel	Servidor local sobre Bottle y Gevent; comunicación Python–JavaScript mediante WebSocket local; funciones expuestas con el decorador <code>@eel.expose</code> ; biblioteca cliente servida en <code>/eel.js</code> ; apertura del navegador en modo aplicación (Chrome o Chromium, con Edge como alternativa en Windows); opción de puerto 0 para asignación automática; licencia MIT [16, 17]

2.2.3. Historia y evolución

Electron. Nació como *Atom Shell*, base del editor Atom de GitHub, que salió en beta pública en abril de 2014. Se concibió como alternativa a node-webkit y al Chromium Embedded Framework, con la idea de integrar Node.js y Chromium. Fue adoptado pronto por Slack, GitKraken y WebTorrent, y se renombró Electron; la versión 1.0 llegó en 2016. Posteriormente su titularidad pasó de GitHub a la OpenJS Foundation [13]. En septiembre de 2026 la rama estable es la 44.x [14].

Eel. Proyecto de código abierto publicado originalmente por Chris Knott en GitHub y hoy alojado bajo la organización *python-eel* [16]. Las versiones publicadas en PyPI son de la serie 0.x; una fuente de seguimiento de paquetes registra la 0.16.0 en febrero de 2023 y otra lista la 0.18.2 como la más reciente [18, 19]. Existen bifurcaciones (*forks*) que declaran retomar o ampliar el proyecto [20]; conviene verificar el estado de mantenimiento antes de depender de él en un producto.

2.2.4. Ventajas y desventajas

Tecnología	Ventajas	Desventajas
Electron	Un solo código para tres sistemas operativos; interfaz web moderna; gran ecosistema; adopción por productos conocidos [13]	Mayor consumo de recursos que una aplicación nativa (crítica recurrente); cada aplicación incluye Chromium y Node.js
Eel	Reduce el código repetitivo para comunicar JavaScript con Python; acceso a todo el ecosistema Python; puesta en marcha rápida [16]	Requiere Chrome, Chromium o Edge instalado; abre un servidor web local por instancia; usa monkey patching de Gevent, que puede interferir con depuradores y con otros modelos de concurrencia [16]; proyecto de alcance declarado reducido

2.2.5. Casos de uso (cuándo usarlo y cuándo no)

Tecnología	Cuándo sí	Cuándo no
Electron	Editores, clientes de mensajería, herramientas de desarrollo, aplicaciones de escritorio con interfaz web rica	Dispositivos con memoria muy limitada; aplicaciones que exigen apariencia y rendimiento estrictamente nativos

Tecnología	Cuándo sí	Cuándo no
Eel	Herramientas internas, utilidades de datos y prototipos con lógica en Python y una interfaz sencilla	Aplicaciones comerciales de larga vida sin verificar el mantenimiento del proyecto; equipos sin navegador Chromium disponible

2.2.6. Casos de aplicación (ejemplos reales en la industria)

Electron: Visual Studio Code, Slack, GitHub Desktop y Discord [13, 15, 14]. Slack figura entre los primeros adoptantes según el propio proyecto [13].

Eel: no se encontraron casos de aplicación industrial documentados en las fuentes consultadas. El proyecto se posiciona para aplicaciones de interfaz cortas y simples [16].

2.3. Backend: Ruby on Rails (modo API)

2.3.1. Definición clara (qué es y qué no es)

Ruby on Rails es un framework web del lado del servidor, escrito en Ruby y publicado bajo licencia MIT, que sigue el patrón modelo–vista–controlador y proporciona estructuras por defecto para base de datos, servicios web y páginas [21]. Su *modo API* restringe el framework a lo necesario para servir datos (sin la pila de vistas), y en este proyecto se declara con `config.api_only = true`. ActiveRecord se omite deliberadamente en `config/application.rb` [1]. *No es* un microenrutador HTTP mínimo, ni un lenguaje, ni obliga a usar ActiveRecord: es una suite de componentes desacoplables.

2.3.2. Características principales

- **Convención sobre configuración** y principio DRY como filosofía de diseño [21].
- **Modo API y Action Cable** (WebSockets integrados), incorporados en la versión 5 [22].
- **Action Cable** exige dos configuraciones: un adaptador de suscripción y los orígenes de petición permitidos [23]. Difunde mensajes mediante publicación/suscripción; las difusiones son una cola en línea sujeta al tiempo: quien no está suscrito no recibe el mensaje si se conecta después [24].
- El servidor Action Cable ejecuta las acciones de canal en un *worker pool* de hilos que, por defecto, se limita a **4 hilos** (parámetro `worker_pool_size`); cada hilo obtiene su propia conexión a Redis o a la base de datos [26].
- Servidor de aplicaciones Puma por defecto y arquitectura sobre Rack.

2.3.3. Historia y evolución

Fecha	Hito
Julio de 2004	David Heinemeier Hansson publica Rails como código abierto, extraído del desarrollo de Basecamp (37signals) [21]
Diciembre de 2005	Versión 1.0 [33]

Fecha	Hito
Octubre de 2007	Apple distribuye Rails con Mac OS X 10.5 Leopard [21]
2013	Rails 4.0 (25 de junio) [21]
2016	Rails 5: modo API y Action Cable [22]
2024	Rails 8 incorpora <i>Solid Cable</i> , <i>Solid Queue</i> y <i>Solid Cache</i> : Solid Cable pasa a ser el adaptador de Action Cable por defecto en producción, con lo que Redis deja de ser una dependencia obligatoria; las aplicaciones existentes conservan su configuración con Redis [27, 28, 29]
2026	Rama estable 8.1 (versión 8.1.3, marzo de 2026) [21]

Desde Rails 7.2, cada versión menor recibe correcciones de errores durante un año y correcciones de seguridad durante dos [30]. Rails 7.1 dejó de recibir soporte de seguridad en octubre de 2025 [31].

2.3.4. Ventajas y desventajas

Aspecto	Descripción
Ventaja: productividad	Alta velocidad de desarrollo gracias a las convenciones; en 2023 Hired la señaló como la habilidad más demandada, con 1,64 veces más solicitudes de entrevista que el promedio del mercado [34, 35]
Ventaja: madurez	Más de veinte años de evolución y adopción por empresas de gran escala [33]
Ventaja: WebSockets integrados	Action Cable evita incorporar un servidor de tiempo real aparte [23]
Desventaja: acoplamiento por convención	Sus convenciones (ActiveRecord, controladores, autocarga) favorecen el acoplamiento; respetar Clean Architecture exige disciplina
Desventaja: concurrencia acotada	El <i>worker pool</i> predeterminado es pequeño y cada hilo consume una conexión a la infraestructura [26]
Desventaja: ciclo de vida corto	Las versiones dejan de recibir seguridad en dos años, lo que obliga a actualizar con frecuencia [30]
Desventaja: adopción menor	Solo 4,7 % de los encuestados de Stack Overflow 2024 usa Rails [52]

2.3.5. Casos de uso (cuándo usarlo y cuándo no)

Cuándo sí	Cuándo no
Aplicaciones web y plataformas SaaS; APIs; productos mínimos viables; funcionalidades de tiempo real de carga moderada con Action Cable	Cómputo numérico o paralelo intensivo; sistemas con muy alta cantidad de conexiones persistentes sin un plan de escalado; latencia crítica de tiempo real estricto

2.3.6. Casos de aplicación (ejemplos reales en la industria)

GitHub, Shopify y Basecamp (donde nació el framework), y en sus inicios Airbnb, Hulu y SoundCloud [33]. Basecamp/HEY ejecuta con Solid Queue unos 20 millones de trabajos diarios [29].

2.4. Persistencia: Redis

2.4.1. Definición clara (qué es y qué no es)

Redis (*Remote Dictionary Server*) es un almacén de estructuras de datos clave-valor en memoria, utilizado como base de datos, caché y bróker de mensajes [36]. Soporta cadenas, *hashes*, listas, conjuntos, conjuntos ordenados, mapas de bits, HyperLogLog, índices geoespaciales y *streams*, y permite ejecutar operaciones atómicas sobre ellos, como incrementar un valor dentro de un hash [37]. *No es* una base de datos relacional (no ofrece SQL ni uniones nativas), ni un almacenamiento pensado para volúmenes que excedan la memoria disponible. La persistencia a disco es opcional y su durabilidad depende de la configuración elegida [37].

2.4.2. Características principales

- Estructuras de datos ricas y operaciones atómicas sobre ellas [37].
- **Guiones Lua** que pueden realizar actualizaciones condicionales sobre varias claves, combinando tipos de datos, de forma atómica. Los scripts EVAL no se nombran, versionan ni persisten, por lo que la aplicación puede necesitar recargarlos tras reiniciar el servidor o conmutar a una réplica; desde la versión 7.0 las *Redis Functions* ofrecen una alternativa [38].
- Transacciones, publicación/suscripción, evicción LRU y distintos niveles de persistencia en disco [37].
- Alta disponibilidad mediante Redis Sentinel y particionado automático mediante Redis Cluster [37].

2.4.3. Historia y evolución

Fecha	Hito
Febrero de 2009	Salvatore Sanfilippo (“antirez”) lo crea al intentar mejorar la escalabilidad de una <i>startup</i> italiana con un analizador de registros web en tiempo real [36, 40]
2015	Sanfilippo se incorpora a Redis Labs [40]
2020	Deja de ser el mantenedor único [36]

Fecha	Hito
Marzo de 2024	Redis Ltd. cambia la licencia BSD de las versiones 7.4 en adelante a la doble licencia RSALv2/SSPL, ambas no aprobadas como código abierto; la Linux Foundation bifurca la última versión BSD como <i>Valkey</i> [36]
Finales de 2024	Sanfilippo regresa a Redis [39]
Mayo de 2025	Redis 8.0 añade AGPLv3 como tercera opción de licencia [36, 39]

2.4.4. Ventajas y desventajas

Aspecto	Descripción
Ventaja: latencia	Todo el conjunto de datos vive en memoria, con operaciones que se completan en microsegundos
Ventaja: atomicidad	Los comandos individuales son atómicos, y con guiones Lua se puede extender la atomicidad a varias claves [37, 38]
Ventaja: estructuras adecuadas	Hashes y sets permiten resolver contadores y unicidad con costo constante
Desventaja: memoria	El tamaño del conjunto de datos queda limitado por la RAM disponible
Desventaja: durabilidad	Depende de la configuración de persistencia; ante una caída puede perderse información no persistida [37]
Desventaja: modelado manual	Sin SQL ni uniones; el diseño de claves e índices recae en el equipo
Desventaja: licencia	El cambio de licencia de 2024 generó bifurcaciones (Valkey) y obliga a revisar los términos según la versión usada [36]

2.4.5. Casos de uso (cuándo usarlo y cuándo no)

Cuándo sí	Cuándo no
Cachés; contadores y tableros de puntuación; sesiones; colas de trabajos; publicación/suscripción; conteos concurrentes	Almacén principal de datos que exigen durabilidad transaccional estricta; datos históricos voluminosos; consultas relacionales complejas

2.4.6. Casos de aplicación (ejemplos reales en la industria)

Existe documentación pública de ingeniería sobre el uso de Redis en Twitter (escalado), Slack (cola de trabajos), Instagram (almacenamiento de pares clave-valor), Twitch (arquitectura de chat), Heroku (flota de Redis) y GitHub, cuyo artículo lleva por título *Moving persistent data out of Redis*; estos casos están recopilados en un listado de lecturas sobre escalabilidad [41]. GitHub e Instagram figuran además entre los primeros adoptantes del proyecto [40].

2.5. Protocolo de integración: WebSockets

2.5.1. Definición clara (qué es y qué no es)

WebSocket es un protocolo de comunicación que proporciona un canal bidireccional (*full duplex*) sobre una única conexión TCP. Fue estandarizado por la IETF como RFC 6455 en diciembre de 2011 [42, 43]. Consta de un apretón de manos de apertura y de un formato básico de mensajes por tramas sobre TCP [42]. *No es* HTTP: su única relación con HTTP es que el apretón de manos se interpreta como una solicitud de actualización de protocolo [43]. Tampoco es un sistema de colas ni ofrece por sí solo confirmaciones de entrega a nivel de aplicación; esas garantías deben construirse encima.

2.5.2. Características principales

- Conexión persistente y bidireccional: el servidor puede enviar datos al cliente sin que este los solicite [46].
- Esquemas `ws://` y `wss://`; el apretón de manos inicial usa HTTP [43].
- Modelo de seguridad basado en el origen (*origin*) [42].
- Tramas de texto, binarias y de control [42].
- Extensiones posteriores: WebSocket sobre HTTP/2 (RFC 8441) y sobre HTTP/3 (RFC 9220) [44].
- Estado en el servidor: al ser conexiones de larga duración, escalar horizontalmente es más complejo que con HTTP sin estado [46].

2.5.3. Historia y evolución

Históricamente, las aplicaciones que necesitaban comunicación bidireccional (mensajería instantánea, juegos) abusaban de HTTP mediante sondeo periódico y llamadas separadas para enviar datos hacia el servidor [42]. El protocolo se propuso en 2008 dentro de la especificación HTML5 (Ian Hickson y Michael Carter) y pasó de W3C y WHATWG a la IETF en febrero de 2010; Google Chrome 4 fue el primer navegador en soportarlo por defecto (diciembre de 2009) [43, 45]. El RFC 6455 se publicó en diciembre de 2011, con Ian Fette (Google) y Alexey Melnikov (Isode) como autores [42]. El soporte se completó en los demás navegadores a lo largo de 2012 [45].

2.5.4. Ventajas y desventajas

Aspecto	Descripción
Ventaja: baja latencia	Elimina el sondeo repetido y permite el envío inmediato desde el servidor [42, 46]
Ventaja: bidireccionalidad	Cliente y servidor envían datos de forma independiente sobre la misma conexión
Desventaja: estado	Conexiones abiertas que ocupan recursos del servidor y complican el escalado [46]
Desventaja: mensajes perdidos	Con Action Cable, un cliente que se reconecta no recibe lo difundido mientras estuvo desconectado [24]

Aspecto	Descripción
Desventaja: seguridad	Requiere validar el origen y autenticar la conexión; en Action Cable el servidor no tiene acceso a la sesión, solo a las cookies [23, 25]
Desventaja: infraestructura	Los proxies y balanceadores deben soportar la actualización de protocolo

2.5.5. Casos de uso (cuándo usarlo y cuándo no)

Cuándo sí	Cuándo no
Comunicación en tiempo real de doble vía: juegos en línea, chats y actualizaciones en vivo [46]; votaciones y tableros de control	Consultas esporádicas, recursos estáticos, reportes por lotes, y escenarios donde la ausencia de estado facilita el escalado y HTTP es suficiente [46]

2.5.6. Casos de aplicación (ejemplos reales en la industria)

- **Discord:** su *Gateway* para eventos en tiempo real usa conexiones WebSocket persistentes que la aplicación debe mantener con latidos periódicos [47].
- **Slack:** ofrece *Socket Mode*, basado en WebSocket, para recibir eventos sin un punto de conexión HTTP público [46].

2.6. Contenerización: Docker y Docker Compose

2.6.1. Definición clara (qué es y qué no es)

Docker es un conjunto de productos que utiliza virtualización a nivel de sistema operativo para entregar el software en paquetes llamados contenedores; su componente central, Docker Engine, ejecuta y gestiona esos contenedores [48]. Docker Compose permite describir y levantar en conjunto los contenedores de una aplicación mediante un archivo declarativo, como el `docker-compose.yml` del repositorio. *No es* una máquina virtual completa (comparte el núcleo del sistema anfitrión), ni un orquestador de clústeres, ni una garantía de seguridad por sí mismo.

2.6.2. Características principales

- Empaqueta la aplicación con sus dependencias en una unidad estándar; cada instancia se ejecuta en el mismo entorno [6].
- Docker Engine para Linux se distribuye bajo licencia Apache-2.0; Docker Desktop tiene un acuerdo de licencia de usuario final propio [48].
- Los datos que deben sobrevivir al contenedor se conservan en volúmenes; el repositorio usa el volumen `redis_data` [1].

2.6.3. Historia y evolución

Docker nació como proyecto interno de dotCloud, una empresa de plataforma como servicio, iniciado por Solomon Hykes [48]. Se presentó públicamente en marzo de 2013, durante PyCon, y se liberó

como código abierto bajo licencia Apache 2.0 [50, 48]. En 2014, con la versión 0.9, sustituyó LXC por su propia biblioteca *libcontainer*, escrita en Go [49]. En 2015 se lanzó el Open Container Project con distintos fabricantes [51]. La rama actual del motor es la 29.x [48].

2.6.4. Ventajas y desventajas

Aspecto	Descripción
Ventaja: reproducibilidad	Entorno idéntico entre desarrollo y ejecución [6]
Ventaja: despliegue simple	Un solo comando levanta backend y base de datos
Ventaja: estándar de la industria	Es la herramienta de compilación y prueba más usada entre los profesionales encuestados (58,7 %) [52]
Desventaja: complejidad operativa	Exige gestionar imágenes, redes, volúmenes y actualizaciones de las imágenes base
Desventaja: datos con estado	Los servicios con persistencia requieren volúmenes y respaldo explícitos
Desventaja: licencias	Docker Desktop tiene condiciones de licencia propias [48]

2.6.5. Casos de uso (cuándo usarlo y cuándo no)

Cuándo sí	Cuándo no
Entornos de desarrollo reproducibles; integración y entrega continuas; aplicaciones multicontenedor; despliegue de servicios sin estado o con volúmenes	Aplicaciones que requieren acceso directo al hardware o interfaces gráficas nativas (como el cliente de escritorio de este proyecto); entornos con un núcleo distinto al del anfitrión

2.6.6. Casos de aplicación (ejemplos reales en la industria)

La aplicación de referencia *eShopOnWeb* de Microsoft se ejecuta como contenedor con Docker Compose [6]. Docker es utilizado por el 58,7 % de los desarrolladores profesionales en la encuesta de Stack Overflow 2024, y Kubernetes por el 22 % [52].

2.7. Relación entre el estilo y las tecnologías seleccionadas

2.7.1. Cómo se materializa el estilo en el repositorio

Círculo del estilo	Implementación	Tecnología involucrada
Entidades	Poll, Option, Vote y Core::Errors	Ruby puro, sin ActiveRecord (omitido en application.rb)
Casos de uso	RegisterVote, GetPollResults	Ruby puro
Adaptador de entrada (WebSocket)	VotingChannel	Action Cable sobre Rails
Adaptador de entrada (HTTP)	PollsController	Rails en modo API
Adaptador de salida	RedisVoteRepository	Redis (SADD, HINCRBY, HSET, HGETALL)
Capa externa	main.py, web/	Eel, Chromium y la biblioteca websockets
Infraestructura de ejecución	docker-compose.yml	Docker Compose: red bridge, volumen redis_data, Redis con -appendonly yes

Adicionalmente, `application.rb` agrega `app/core` y `app/infrastructure` a las rutas de autocarga, de modo que las capas del estilo son carpetas reconocidas por Rails [1]. El cliente Python implementa directamente el subprotocolo de ActionCable (mensaje de bienvenida, `subscribe`, `message` y `ping`) sobre la biblioteca `websockets` [1].

2.7.2. Por qué encajan

1. **Un mismo caso de uso, dos mecanismos de entrega.** `VotingChannel` y `PollsController` invocan el mismo `RegisterVote`: es la demostración práctica de la independencia respecto del canal de entrega [1].
2. **Redis detrás de un repositorio.** El acceso a Redis está concentrado en `RedisVoteRepository`, lo que ubica el detalle de persistencia en la periferia.
3. **Cliente desacoplado.** El terminal de escritorio se comunica solo por mensajes JSON sobre WebSocket y no comparte código con el backend.
4. **Operaciones atómicas de Redis alineadas con las reglas de negocio.** `SADD` resuelve la unicidad de votante e `HINCRBY` el conteo concurrente [37].
5. **Redis como bus de Action Cable.** La configuración de `cable.yml` usa el adaptador Redis, que mantiene sincronizados los mensajes entre procesos de servidor [23].

2.7.3. Tensiones detectadas entre el estilo y el stack

Las siguientes observaciones se verificaron en el código y en la documentación oficial; son insumos para el análisis posterior.

Punto	Observación verificada	Implicación
Regla de Dependencia	<code>register_vote.rb</code> y <code>get_poll_results.rb</code> declaran <code>Infrastructure::Repositories::RedisVoteRepository.new</code> como valor por defecto, y <code>RegisterVote</code> usa <code>ActionCable.server</code> como difusor por defecto; <code>VotingChannel</code> y <code>PollsController</code> lo instancian sin argumentos	El núcleo nombra a la infraestructura y al framework: la inyección es posible, pero no se aplica en la práctica
Atomicidad del voto	<code>record_vote</code> ejecuta <code>SADD</code> , <code>HINCRBY</code> y <code>HSET</code> como comandos separados	Cada comando es atómico, la secuencia no. Redis permite unificarla con un guion Lua [38], considerando su recarga tras reinicios
Independencia del motor	<code>find_options</code> y <code>calculate_results</code> asumen el esquema de hashes de Redis	Cambiar de motor exigiría un puerto explícito
Electron vs. Eel	El cliente se ejecuta en modo <code>chrome/edge/default</code> ; no incluye <code>Electron</code>	Debe declararse; Eel admite el modo <code>electron</code> [16]
Concurrencia	<code>Action Cable</code> ejecuta las acciones de canal en un <i>worker pool</i> de 4 hilos por defecto [26]; el repositorio usa un único cliente global <code>REDIS_CLIENT</code>	Dimensionar el <i>pool</i> y evaluar un <i>pool</i> de conexiones
Orígenes permitidos	En desarrollo se permite cualquier origen y se desactiva la protección contra falsificación de solicitudes; <code>CORS</code> usa <code>origins "*"</code> ; la variable <code>ALLOWED_ORIGINS</code> de <code>Compose</code> no se lee en el código; no existe configuración de producción	<code>Action Cable</code> exige orígenes permitidos configurados [23]
Versiones	<code>Dockerfile</code> usa Ruby 3.2, cuyo fin de vida fue el 31 de marzo de 2026 [32]; el <code>Gemfile</code> fija Rails 7.1, sin soporte de seguridad desde octubre de 2025 [31]	Riesgo de mantenimiento; planificar actualización
Imagen de Redis	<code>redis:7-alpine</code> sigue la serie 7.x, y desde la 7.4 la licencia cambió [36]	Verificar la versión efectiva y su licencia; Valkey es la alternativa BSD
Redis como único almacén	El sistema no usa una base durable complementaria	Coherente con la latencia buscada, pero con riesgo de durabilidad [37]

2.8. Qué tan común es el stack designado

2.8.1. Prevalencia individual de cada tecnología

Datos de la encuesta de desarrolladores de Stack Overflow 2024 (65 437 respuestas) [52]:

Tecnología	Todos	Profesionales	Categoría
Python	51 %	46,9 %	Lenguaje
Ruby	5,2 %	5,8 %	Lenguaje
Ruby on Rails	4,7 %	5,2 %	Framework web
Redis	20 %	22,8 %	Base de datos
Electron	6,5 %	6,3 %	Framework/biblioteca
Tauri (alternativa)	2,4 %	2,1 %	Framework/biblioteca
Docker	53,9 %	58,7 %	Herramienta
Kubernetes (referencia)	19,4 %	22 %	Herramienta

Eel no figura entre las opciones de la encuesta. WebSocket es un protocolo de plataforma, no una tecnología encuestada. Docker es la herramienta más usada por los profesionales y la más admirada (78 %), y Redis es la base de datos que el 28 % de los usuarios de PostgreSQL desea usar a continuación [52].

2.8.2. Relación entre las tecnologías

Combinación	Grado de relación	Evidencia
Rails + WebSockets	Integración nativa	Action Cable forma parte de Rails desde la versión 5 [22]
Action Cable + Redis	Documentada y habitual	Redis es un adaptador de suscripción oficial y ha sido el predeterminado de producción [23, 24]. En Rails 8 el predeterminado pasa a Solid Cable, aunque los proyectos existentes conservan Redis [28, 29]
Redis + Docker	Compatible	El repositorio ejecuta Redis desde la imagen <code>redis:7-alpine</code> orquestada con Compose [1]
Rails + Docker	Compatible	El repositorio empaqueta el backend en un contenedor con un <code>Dockerfile</code> propio basado en Ruby 3.2 [1]
Python + WebSockets	Viable	El cliente del repositorio usa la biblioteca <code>websockets</code> ; Eel depende además de <code>bottle-websocket</code> para su canal local [1, 18]
Electron + Python (Eel)	Poco habitual	Electron se asocia habitualmente con Node.js; Eel es un nicho para desarrolladores Python [16, 14]

2.8.3. Conclusión

El conjunto **Rails + Redis + WebSockets** es una combinación consolidada y documentada por el propio framework, aunque Rails 8 ya no exige Redis para Action Cable. **Docker** es una práctica estándar en la industria. La pieza menos común es **Electron + Eel**: su adopción individual es baja, no hay casos industriales documentados de Eel y su modo Electron no se usa en la implementación. El stack completo es un sistema distribuido heterogéneo (Ruby en el servidor, Python en el cliente), en el que el contrato de mensajes JSON sobre WebSocket es la frontera que mantiene el desacoplamiento. Ninguna de las tecnologías elegidas es incompatible con el estilo; las tensiones detectadas provienen de la forma en que se aplicó el estilo, no del stack en sí.

3. Fase de diseño: matrices de análisis arquitectónico

Las siguientes matrices corresponden a la fase de diseño del proyecto. A diferencia de la investigación de la Sección 2, aquí el objetivo no es reunir evidencia externa sino tomar postura: valorar si el estilo arquitectónico asignado soporta, limita o resulta neutral frente a un conjunto de criterios de diseño, sin extenderse en la justificación con evidencia de código que corresponde a fases posteriores.

3.1. Matriz 1: Atributos de calidad vs Estilo arquitectónico

La Tabla 28 cruza las nueve características de calidad de ISO/IEC 25010:2023 con el estilo Clean Architecture. Cada fila se clasifica como *Soporta* cuando el estilo favorece esa característica por su propia estructura, *Limita* cuando la restringe o le añade costo, y *Neutral* cuando el estilo no la determina y la característica depende de decisiones de infraestructura ajenas al patrón.

Tabla 28: Atributos de calidad (ISO/IEC 25010:2023) frente al estilo Clean Architecture

Atributo de Calidad	¿El estilo lo soporta o limita?	Justificación Técnica
Functional suitability	Soporta	Cada regla de negocio se modela como un caso de uso aislado y explícito, lo que facilita cubrir el requisito funcional completo sin mezclarlo con detalles de framework o de persistencia.
Performance efficiency	Limita (leve)	La separación en capas introduce una indirección entre la solicitud y su procesamiento; el costo es marginal frente a la latencia de red o de E/S, pero existe por diseño.
Compatibility	Soporta	Los adaptadores de entrada y salida aíslan el núcleo de los formatos y protocolos externos, de modo que pueden coexistir varios canales de entrada sin que el dominio dependa de ninguno en particular.

Atributo de Calidad	¿El estilo lo soporta o limita?	Justificación Técnica
Interaction capability	Neutral	El estilo no define nada sobre la interfaz de usuario; solo garantiza que esta pueda sustituirse sin alterar las reglas de negocio, sin aportar por sí mismo usabilidad o accesibilidad.
Reliability	Neutral	El estilo permite expresar errores de dominio de forma explícita, pero la tolerancia a fallos, la recuperación y la disponibilidad dependen de las decisiones de infraestructura, no de la organización en capas.
Security	Neutral	Las reglas de autorización pueden concentrarse en los casos de uso, pero el estilo no impone ningún mecanismo de autenticación, cifrado o control de acceso; esas decisiones quedan fuera de su alcance.
Maintainability	Soporta	Es el objetivo central del estilo: la Regla de Dependencia aísla cada responsabilidad en su propio círculo, de modo que un cambio en un detalle externo no se propaga al núcleo.
Flexibility	Soporta	La Inversión de Dependencias permite sustituir infraestructura, reemplazar un adaptador o escalar instancias sin modificar el núcleo, siempre que se respete el contrato definido por el puerto.
Safety	No aplica	No existe riesgo físico asociado al software en este dominio; la característica queda fuera del alcance del análisis.

3.2. Matriz 2: Principios de diseño vs Estilo arquitectónico

La Tabla 29 verifica el cumplimiento de doce principios de diseño de software (los cinco principios SOLID, KISS, DRY, YAGNI, PoLA, la Ley de Demeter, el antipatrón STUPID y Composición sobre Herencia) frente a la estructura que impone Clean Architecture, distinguiendo un cumplimiento pleno (*Sí*), uno que depende de la disciplina del equipo (*condicionado*), uno logrado solo en parte (*Parcial*) y uno que el estilo no garantiza (*No*).

Tabla 29: Principios de diseño de software frente al estilo Clean Architecture

Principio	¿El estilo lo cumple?	Justificación Técnica
SRP (Responsabilidad Única)	Sí	Cada círculo agrupa un único motivo de cambio: las entidades cambian por reglas de negocio, los casos de uso por reglas de aplicación y los adaptadores por decisiones de infraestructura.
OCP (Abierto/Cerrado)	Sí	Un nuevo mecanismo de entrega o una nueva implementación de persistencia se añaden creando un adaptador que satisface el puerto existente, sin modificar el núcleo ya escrito.
LSP (Sustitución de Liskov)	Sí (condicionado)	El puerto define un contrato que cualquier adaptador debe cumplir; el estilo lo exige, pero su cumplimiento real depende de que cada implementación respete ese contrato al pie de la letra.
ISP (Segregación de Interfaces)	Sí (condicionado)	El estilo favorece puertos pequeños y específicos por caso de uso; el riesgo aparece si se diseña un único repositorio con demasiados métodos que ningún consumidor usa en su totalidad.
DIP (Inversión de Dependencias)	Sí	Es el mecanismo que sostiene toda la arquitectura: el núcleo declara la abstracción que necesita y la infraestructura se adapta a ella, invirtiendo la dependencia habitual hacia la base de datos o el framework.
KISS (Simplicidad)	No	El estilo exige capas, puertos, adaptadores y objetos de transferencia de datos; esa estructura añade complejidad accidental que un diseño más directo no tendría.
DRY (No te Repitas)	Parcial	Un caso de uso reutilizado por varios adaptadores de entrada evita duplicar la lógica de negocio; sin embargo, el estilo no evita por sí mismo la duplicación de mapeos entre las estructuras de cada frontera.

Principio	¿El estilo lo cumple?	Justificación Técnica
YAGNI (No lo vas a necesitar)	No	El estilo recomienda anticipar puertos y abstracciones para piezas que aún podrían no cambiar nunca, lo que puede introducir estructura que el proyecto no llega a necesitar.
PoLA (Principio de Mínima Sorpresa)	Sí	Al separar con claridad qué hace cada círculo, un desarrollador puede predecir dónde vive cada responsabilidad y qué comportamiento esperar de cada capa.
Ley de Demeter	Sí	Los casos de uso se comunican directamente con sus colaboradores inmediatos (repositorio, difusor) a través del puerto, sin necesidad de encadenar llamadas a través de objetos intermedios.
STUPID (prevención de antipatronos)	Sí	La Regla de Dependencia y la inyección de colaboradores previenen por diseño el acoplamiento rígido, las dependencias fijas al framework y el código difícil de probar que agrupa STUPID.
Composición sobre Herencia	Sí	Los casos de uso se construyen recibiendo colaboradores por composición (repositorio, difusor) en lugar de heredar comportamiento de clases base, lo que mantiene cada pieza independiente y sustituible.

3.3. Matriz 3: Tácticas arquitectónicas vs Estilo y stack

La Tabla 30 evalúa un conjunto de tácticas arquitectónicas de calidad (Bass, Clements y Kazman) frente al estilo Clean Architecture y frente al stack tecnológico asignado por separado, ya que una táctica puede estar cubierta por el estilo, por una tecnología concreta, por ambos o por ninguno. Las filas marcadas con **Amerita ADR** señalan decisiones que el equipo debe documentar formalmente antes de avanzar a la implementación.

Tabla 30: Tácticas arquitectónicas frente al estilo Clean Architecture y al stack tecnológico

Táctica	¿El estilo y el stack la soportan?	Justificación Técnica
Mantener datos en memoria	Sí (stack)	Redis conserva el estado activo en RAM, lo que evita el costo de E/S a disco en cada lectura o escritura frecuente.
Reducir la sobrecarga de comunicación	Sí (stack)	WebSockets mantiene una conexión persistente entre cliente y servidor, evitando el costo de abrir y cerrar peticiones HTTP repetidas para cada actualización.
Introducir concurrencia	Parcial (stack)	El framework del backend ejecuta las acciones en un grupo acotado de hilos; el estilo, al mantener los casos de uso sin estado propio, no impide escalar ese grupo, pero tampoco lo resuelve por sí solo. Amerita ADR para fijar el tamaño del grupo de concurrencia y su justificación frente a la carga esperada.
Gestionar recursos mediante un pool	No (stack, tal como está)	Un cliente único hacia el almacén en memoria no reparte la carga entre conexiones concurrentes. Amerita ADR que decida entre un pool de conexiones o un cliente compartido, con sus compensaciones de simplicidad frente a rendimiento.
Detección de fallos	Parcial (stack)	Un punto de verificación de salud informa si el proceso principal responde, pero no necesariamente si sus dependencias externas (persistencia, mensajería) también lo hacen.
Recuperación ante fallos	Parcial (stack)	La persistencia configurada en el almacén en memoria permite reconstruir el estado tras un reinicio; la política de reconexión del cliente determina cuánto tarda el sistema en volver a operar.

Táctica	¿El estilo y el stack la soportan?	Justificación Técnica
Prevención mediante atomicidad	Limita (estilo + stack)	Cada operación individual sobre el almacén es atómica, pero una secuencia de varias operaciones relacionadas no lo es por defecto. Amerita ADR que decida el mecanismo (script atómico, transacción) para proteger esa secuencia como una sola unidad.
Autenticar actores	No (stack, tal como está)	El canal de comunicación en tiempo real no valida por defecto la identidad de quien se conecta. Amerita ADR que defina el mecanismo de autenticación antes de exponer el sistema fuera de un entorno controlado.
Limitar la exposición	No (stack, tal como está)	Sin restricción de orígenes ni cifrado en tránsito, cualquier cliente en la red podría conectarse. Amerita ADR que fije la política de orígenes permitidos y el uso de conexión cifrada.
Usar un intermediario	Sí (estilo)	El patrón Repositorio actúa como intermediario entre el caso de uso y el motor de persistencia, permitiendo cambiar este último sin tocar la regla de negocio.
Restringir las dependencias	Sí (estilo)	La Regla de Dependencia limita explícitamente qué círculo puede conocer a cuál, reduciendo el acoplamiento entre el negocio y los detalles técnicos.
Inyección de dependencias	Sí (estilo)	Los casos de uso reciben sus colaboradores (repositorio, difusor) desde afuera, lo que permite sustituirlos por versiones de prueba sin levantar infraestructura real.

3.4. Matriz 4: Mercado laboral vs Estilo y stack

La Tabla 31 estima, para cada tecnología del stack, su porcentaje de uso en la industria, el nivel de demanda laboral asociado y un rango salarial de referencia tanto para trabajo remoto en Latinoamérica como para Estados Unidos. Este cruce complementa las matrices anteriores con un criterio de empleabilidad: además de si una tecnología encaja técnicamente en el estilo, importa qué tan buscada está en el mercado.

Tabla 31: Mercado laboral: demanda y remuneración por tecnología del stack

Tecnología / Componente	Tipo / Rol	% de Uso en la Industria	Demanda Laboral	Salario Remoto LatAm (USD/año)	Salario EE. UU. (USD/año)
Docker & Docker Compose	Contenerización / DevOps	~53 % (Herramienta #1 de despliegue)	Muy Alta (Estándar de la industria)	\$45 000 – \$95 000	\$120 000 – \$180 000
Redis 7.x	Persistencia en memoria / Caché	~20 %–22 % (Top 5 bases de datos)	Muy Alta (Crítica en tiempo real)	\$50 000 – \$100 000	\$130 000 – \$190 000
Clean Architecture	Patrón Arquitectónico	~18 %–25 % (Proyectos Mid/Senior)	Alta (Requisito clave de seniority)	\$60 000 – \$115 000	\$145 000 – \$210 000
WebSockets (Action Cable)	Comunicación Bidireccional	~20 % (Sistemas en tiempo real)	Media-Alta (Nicho especializado)	\$45 000 – \$85 000	\$120 000 – \$170 000
Ruby on Rails (API pura)	Backend Framework	~4,7 % (Ecosistema Rails global)	<1 % modo API pura sin ActiveRecord; media en volumen / alta en paga	\$50 000 – \$95 000	\$125 000 – \$180 000
Electron	Frontend Escritorio	~6 %–8 % (Líder en desktop multi- plataforma)	Media (Aplicaciones específicas)	\$40 000 – \$75 000	\$110 000 – \$160 000

Tecnología / Componente	Tipo / Rol	% de Uso en la Industria	Demanda Laboral	Salario Remoto LatAm (USD/año)	Salario EE. UU. (USD/año)
Python (Eel)	GUI / Puente Python-Web	<0,1 % (Herramienta experimen- tal/nicho)	Muy Baja (Inexistente como requisito formal)	\$35 000 – \$65 000*	\$90 000 – \$130 000*

4. Diagramas de arquitectura

Esta sección reúne las vistas gráficas del sistema: el contexto y los contenedores según el modelo C4, un diagrama de alto nivel del flujo de comunicación, el diagrama de secuencia del caso de uso principal, el modelo entidad-relación de los datos y el diagrama de despliegue del entorno local. Cada figura se acompaña de una descripción que la conecta con el código y las decisiones ya documentadas en las secciones anteriores.

4.1. Diagrama de contexto (C4 — Nivel 1)

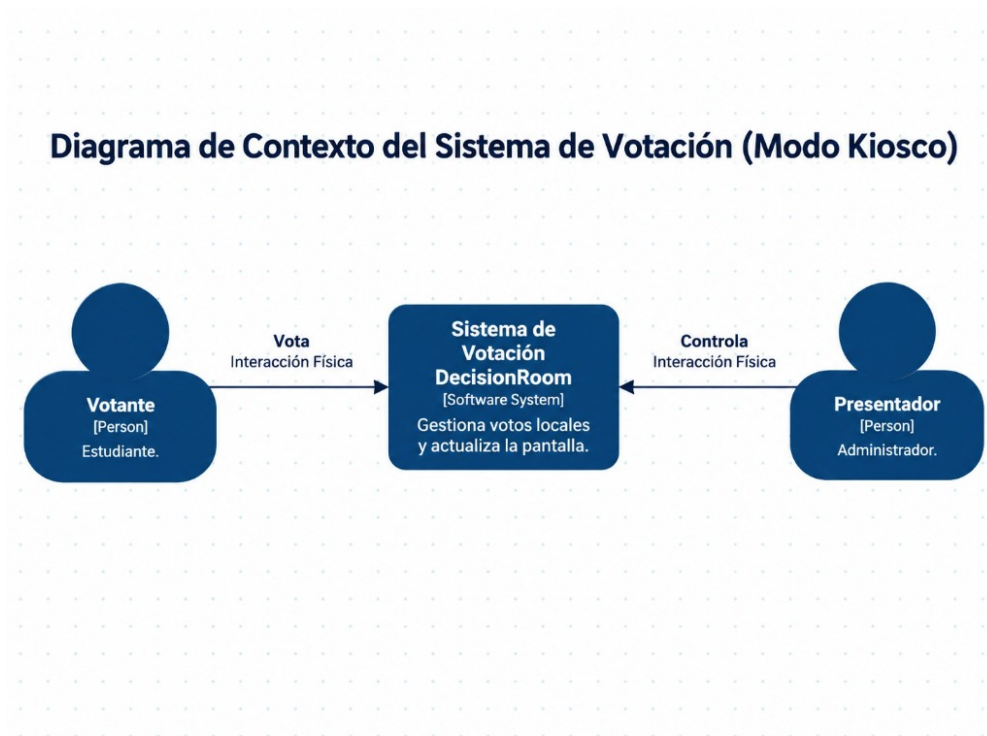


Figura 1: Diagrama de contexto del sistema de votación DecisionRoom, en modo kiosco

La Figura 1 sitúa al sistema *DecisionRoom* como una única caja negra frente a sus dos actores humanos. El **Votante** interactúa físicamente con el kiosco para emitir su voto, mientras que el **Presentador** lo controla como administrador de la encuesta (abrirla, cerrarla, reiniciarla). Al tratarse de un despliegue en modo kiosco, ambos actores comparten el mismo terminal físico en lugar de conectarse de forma remota, y el sistema se describe en el diagrama como responsable de gestionar los votos localmente y de actualizar la pantalla en tiempo real.

4.2. Diagrama de contenedores (C4 — Nivel 2)

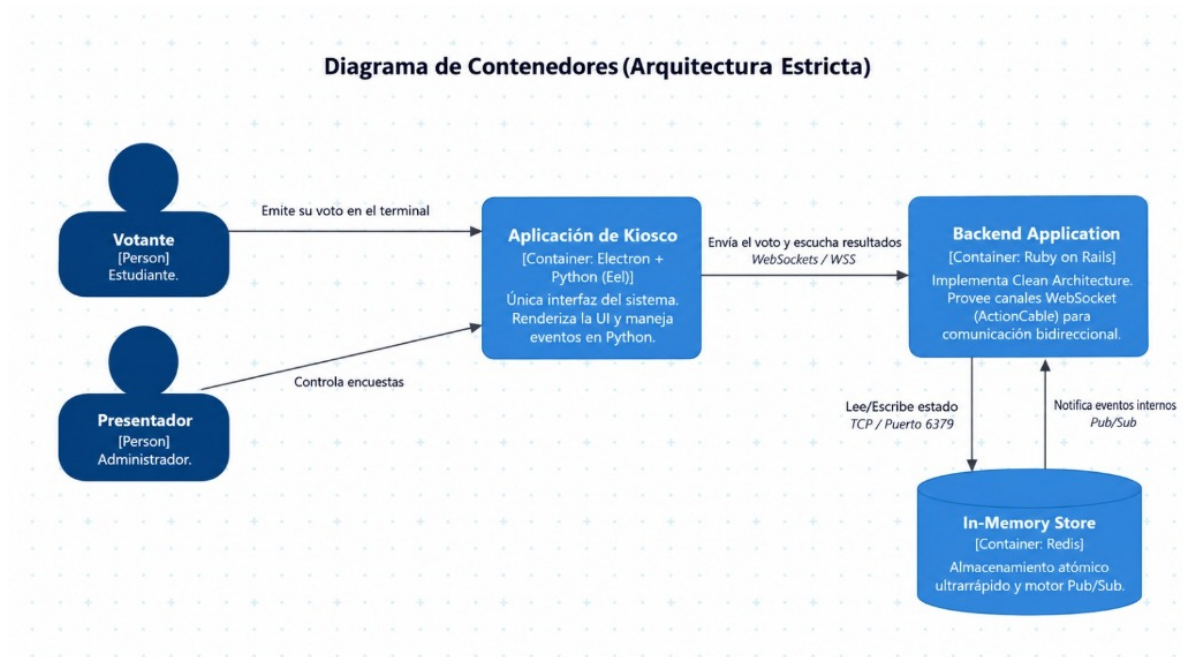


Figura 2: Diagrama de contenedores: aplicación de kiosco, backend Rails y almacén Redis

La Figura 2 descompone el sistema en sus tres contenedores desplegables. La **Aplicación de Kiosco** (Electron + Python/Eel) es la única interfaz del sistema y renderiza la UI en Python; envía el voto y escucha resultados sobre **WebSockets/WSS**. La **Backend Application** (Ruby on Rails) implementa Clean Architecture y expone canales WebSocket mediante Action Cable para la comunicación bidireccional; corresponde a las clases `VotingChannel`, `PollsController` y a los casos de uso descritos en la Sección 2. El **In-Memory Store** (Redis) se describe como almacenamiento atómico ultrarrápido y motor Pub/Sub: el backend lee y escribe su estado por TCP en el puerto 6379, y Redis notifica eventos internos de vuelta al backend por el canal de publicación/suscripción, el mismo mecanismo que sostiene la difusión de resultados documentada en la Sección 2.

4.3. Diagrama de alto nivel (arquitectura general)



Figura 3: Diagrama de alto nivel del sistema de votación: arquitectura general y flujo principal de comunicación

La Figura 3 traza el flujo de extremo a extremo en cuatro bloques. En la **interacción en kiosco**, el votante (estudiante) vota y el presentador (administrador) controla, ambos por interacción física. Esa entrada llega a la **aplicación de escritorio** (Electron + Python), que concentra la interfaz de usuario y la lógica del cliente. La aplicación se comunica en tiempo real por **WebSocket** con el **servidor backend** (Ruby on Rails), que expone la API y el canal WebSocket, aplica la lógica de negocio bajo Clean Architecture y ejecuta las validaciones de dominio; a cambio, el servidor envía los resultados actualizados. Finalmente, el backend lee y escribe en la **base de datos** (Redis) las encuestas, opciones, votos y contadores que sostienen el estado de la votación.

4.4. Diagrama de secuencia: registrar un voto

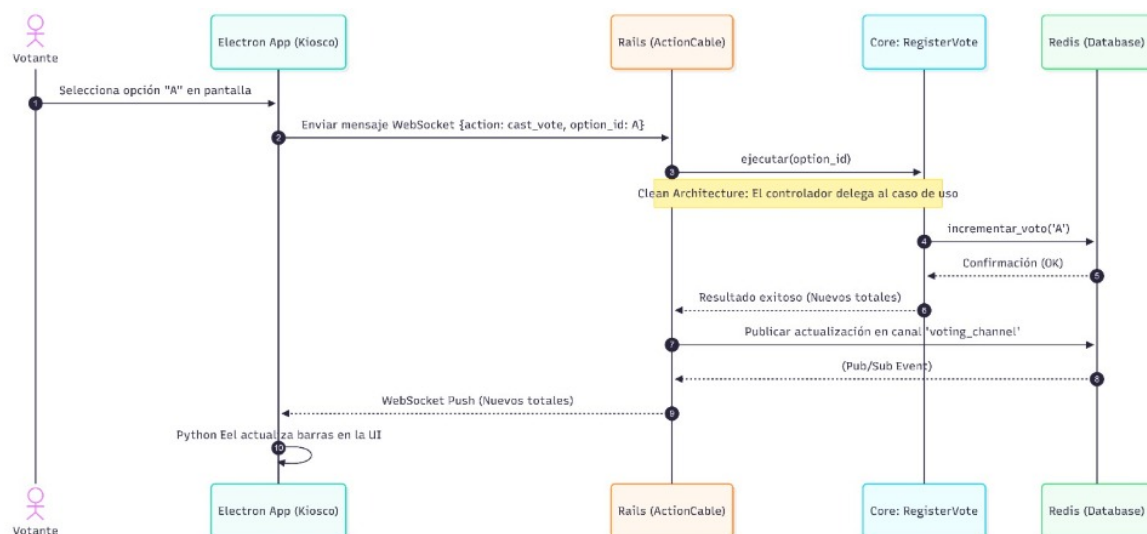


Figura 4: Diagrama de secuencia del caso de uso **RegisterVote**, de la selección del voto a la actualización de la pantalla

La Figura 4 detalla, paso a paso, el recorrido de un voto ya descrito de forma narrativa en la Sección 2. El votante selecciona una opción en la **Electron App**, que empaqueta el evento como un mensaje WebSocket `{action: cast_vote, option_id: A}` hacia **Rails (Action Cable)**. El canal ejecuta `ejecutar(option_id)` sobre **Core: RegisterVote**; la nota resaltada en el propio diagrama subraya el punto central del estilo: “*Clean Architecture: el controlador delega al caso de uso*”, es decir, **VotingChannel** no contiene la regla de negocio, solo la invoca. El caso de uso incrementa el voto en **Redis**, que confirma la operación; con el resultado exitoso y los nuevos totales, Rails publica la actualización en el canal `voting_channel` (el evento Pub/Sub que Redis distribuye), la difunde de vuelta a la Electron App como un WebSocket push, y Python/Eel actualiza las barras de resultados en la interfaz.

4.5. Diagrama entidad–relación

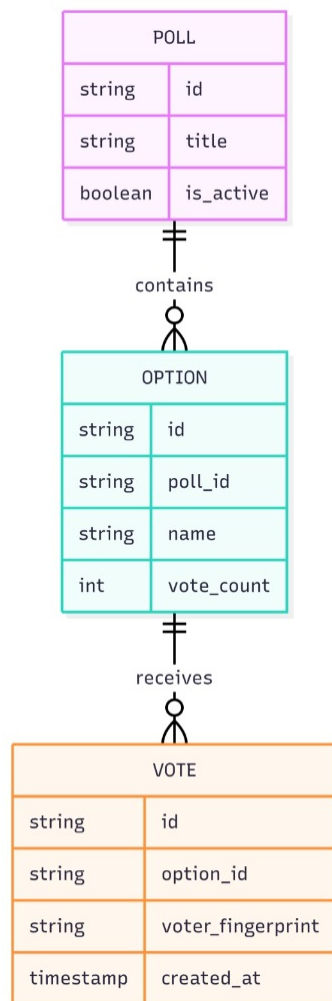


Figura 5: Modelo entidad–relación de las votaciones, sus opciones y los votos registrados

La Figura 5 representa el modelo conceptual detrás de las claves de Redis descritas en la Sección 1.4: una **POLL** (`id`, `title`, `is_active`) contiene una o varias **OPTION** (`id`, `poll_id`, `name`, `vote_count`), y cada opción recibe cero o varios **VOTE** (`id`, `option_id`, `voter_fingerprint`, `created_at`). El campo `voter_fingerprint` es la pieza que sostiene la regla de unicidad de votante implementada con `SADD` sobre el set `poll:{id}:voters`, mientras que `vote_count` corresponde al contador que mantiene `HINCRBY` sobre el hash `poll:{id}:options`. Aunque el diagrama se dibuja con la notación de pie de gallo propia de un modelo relacional, el repositorio no usa una base

relacional: es una traducción conceptual de un esquema que en Redis se implementa como hashes y sets, no como tablas con llaves foráneas reales.

4.6. Diagrama de despliegue

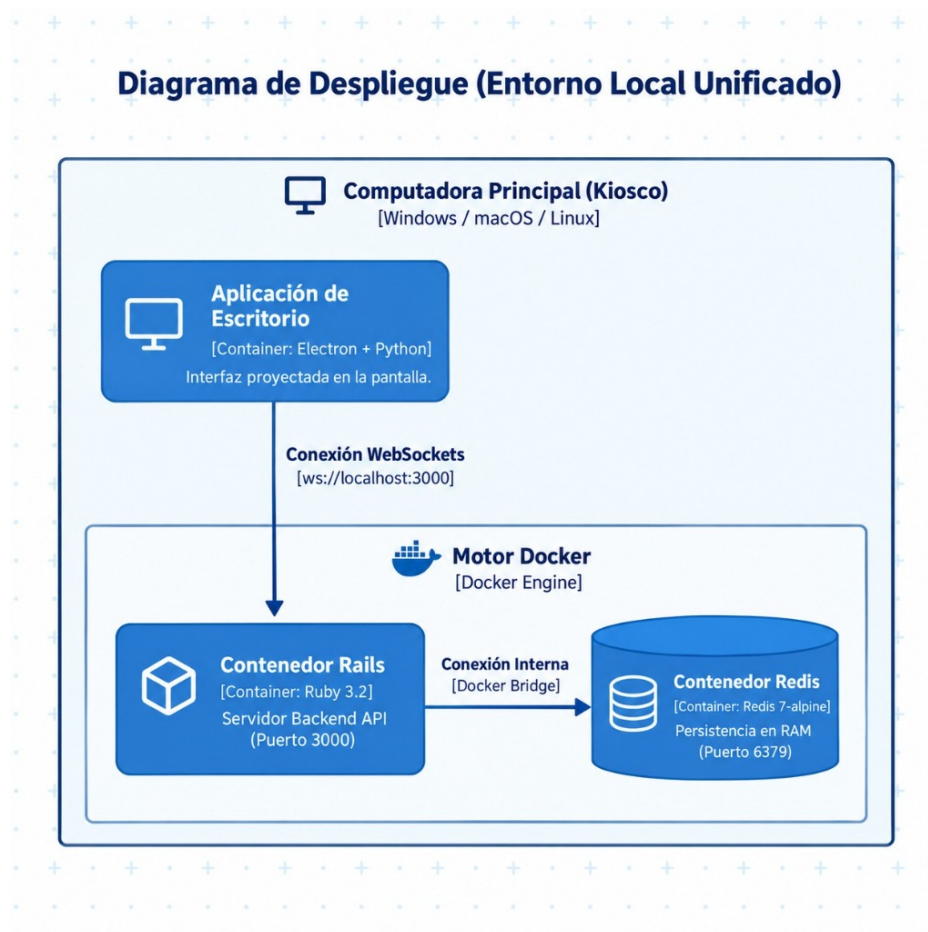


Figura 6: Diagrama de despliegue del entorno local unificado

La Figura 6 documenta el entorno de ejecución tal como lo define `docker-compose.yml`. Sobre la **computadora principal** (Windows, macOS o Linux) corren dos piezas: la **aplicación de escritorio** (contenedor conceptual Electron + Python), cuya interfaz se proyecta en la pantalla del kiosco, y el **motor Docker**, que aloja dos contenedores reales. La aplicación de escritorio se conecta al backend por WebSocket contra `ws://localhost:3000`. Dentro del motor Docker, el **contenedor Rails** (imagen basada en Ruby 3.2) sirve la API backend en el puerto 3000 y se comunica con el **contenedor Redis** (imagen `redis:7-alpine`) por la red interna de Docker (*bridge*), que persiste en RAM sobre el puerto 6379. A diferencia de la aplicación de escritorio, Rails y Redis sí están contenerizados de forma real, consistente con la tensión ya señalada en la Sección 2: el cliente de escritorio queda fuera de Docker porque necesita acceso directo a la pantalla del sistema anfitrión.

5. Lecciones aprendidas

El desarrollo, análisis e integración de los componentes en *DecisionRoom G3* permitió extraer conclusiones fundamentales tanto a nivel de diseño arquitectónico como de interoperabilidad en sistemas distribuidos heterogéneos:

1. Disciplina de Clean Architecture contra las convenciones del framework:

Implementar Clean Architecture sobre Ruby on Rails supuso una fricción inicial significativa, ya que la filosofía idiomática de Rails promueve el acoplamiento (*Convention over Configuration*, uso intensivo de `ActiveRecord` y patrones *Active Record*). Al prescindir intencionalmente de `ActiveRecord` y aislar los casos de uso (`RegisterVote`, `GetPollResults`) y las entidades de dominio en objetos Ruby puros, se incrementó la sobrecarga inicial de diseño. No obstante, este esfuerzo facilitó enormemente la testabilidad aislada de las reglas de negocio, permitiendo validar la lógica de la votación de forma rápida, repetible y completamente independiente del motor de persistencia o del servidor web.

2. Heterogeneidad de lenguajes y rigidez contractual (Python y Ruby):

La coexistencia de dos ecosistemas diferentes en el sistema (un cliente de escritorio en Python mediante Eel y un backend en Ruby) eliminó cualquier posibilidad de compartir código o acoplar librerías cliente-servidor. Esto forzó a formalizar un único idioma común basado en contratos de mensajes JSON estrictos sobre WebSockets (emulando y respetando el ciclo de vida del subprotocolo de Action Cable: bienvenida, suscripción, pings y difusión). Esta separación obligada garantizó un desacoplamiento total en la frontera entre la interfaz y el servidor.

3. El poder y las contrapartidas del almacenamiento en memoria (Redis):

Se comprobó empíricamente que delegar conteos y validaciones concurrentes en comandos atómicos en memoria (`HINCRBY` para contadores y `SADD` para la unicidad de votantes) ofrece un rendimiento y velocidad imbatibles frente a operaciones equivalentes sobre bases de datos relacionales tradicionales. Sin embargo, prescindir de un ORM y de esquemas relacionales automáticos obligó al equipo a diseñar, estructurar y gestionar manualmente los espacios de nombres y el ciclo de vida de las llaves (`poll:{id}:options`, `poll:{id}:voters`, etc.), demandando una mayor rigurosidad en la consistencia de los datos.

4. Acotamiento del alcance a un “Kiosco de Escritorio”:

Frente al riesgo de abordar una arquitectura sobredimensionada, compleja o mixta, acotar la interfaz a una aplicación de kiosco de escritorio dedicada permitió cumplir estrictamente con los requisitos del stack y estabilizar el sistema de punta a punta. Se redujeron puntos de falla distribuidos y se garantizó un ciclo de retroalimentación en tiempo real fluido entre la interacción física y la persistencia en memoria.

Referencias

- [1] KatheBravo. *taller1-arqui-G3* (DecisionRoom G3). Repositorio, commit 2adfe51. <https://github.com/KatheBravo/taller1-arqui-G3>
- [2] R. C. Martin. *The Clean Architecture*. Entrada de blog, 2012. Fecha por contrastar con la publicación original.
- [3] R. C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [4] J. Palermo. *The Onion Architecture*. Serie de entradas de blog, 2008. Fecha por contrastar con la publicación original.
- [5] *Hexagonal architecture (software)*. Wikipedia. [https://en.wikipedia.org/wiki/Hexagonal_architecture_\(software\)](https://en.wikipedia.org/wiki/Hexagonal_architecture_(software))
- [6] Microsoft Learn. *Common web application architectures (.NET)*. <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures>
- [7] Microsoft Learn. *Architectural principles (.NET)*. <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/architectural-principles>
- [8] .NET Blog. *Web Applications with ASP.NET Core Architecture and Patterns guidance*. <https://devblogs.microsoft.com/dotnet/web-apps-aspnetcore-architecture-guidance/>
- [9] ISE Developer Blog. *Next-Level Boilerplate: An Inside Look Into Our .Net Clean Architecture Repo, 2024*. <https://devblogs.microsoft.com/ise/next-level-clean-architecture-boilerplate/>
- [10] S. Smith (Ardalis). *Clean Architecture with ASP.NET Core*. <https://ardalis.com/clean-architecture-asp-net-core/>
- [11] *basic-microservice-example*: ejemplo de cómo Nubank organiza y prueba microservicios (copia pública). <https://github.com/v3gatron/basic-microservice-example>
- [12] R. Nanavaty. *Clean Architecture*. Medium, 13 de octubre de 2023. <https://medium.com/@rudrakshnanavaty/clean-architecture-7c1b3b4cb181>
- [13] Electron. *10 years of Electron*. <https://www.electronjs.org/blog/10-years-of-electron>
- [14] *Electron (software framework)*. Wikipedia. [https://en.wikipedia.org/wiki/Electron_\(software_framework\)](https://en.wikipedia.org/wiki/Electron_(software_framework))
- [15] *Software:Electron (software framework)*. HandWiki. [https://handwiki.org/wiki/Software:Electron_\(software_framework\)](https://handwiki.org/wiki/Software:Electron_(software_framework))
- [16] python-eel. *Eel: A little Python library for making simple Electron-like HTML/JS GUI apps*. <https://github.com/python-eel/Eel>
- [17] *Eel*. PyPI. <https://pypi.org/project/Eel/>
- [18] *PyPI Download Stats: eel*. <https://pypistats.org/packages/eel>
- [19] *eel*. PyUp. <https://pyup.io/packages/pypi/eel/>
- [20] *Paling* (fork de Eel). PyPI. <https://pypi.org/project/Paling>
- [21] *Ruby on Rails*. Wikipedia. https://en.wikipedia.org/wiki/Ruby_on_Rails
- [22] GeeksforGeeks. *Ruby on Rails Tutorial*. <https://www.geeksforgeeks.org/ruby/ruby-on-rails-tutorial/>
- [23] Ruby on Rails Guides. *Action Cable Overview*. https://edgeguides.rubyonrails.org/action_cable_overview.html
- [24] Ruby on Rails Guides v5.2. *Action Cable Overview*. https://guides.rubyonrails.org/v5.2/action_cable_overview.html
- [25] Ruby on Rails Guides v5.0. *Action Cable Overview*. https://guides.rubyonrails.org/v5.0.0/action_cable_overview.html

- [26] *Class: ActionCable::Server::Base*. RubyDoc. <https://rubydoc.info/docs/rails/ActionCable/Server/Base>
- [27] Ruby on Rails Guides. *Ruby on Rails 8.0 Release Notes*. https://guides.rubyonrails.org/8_0_release_notes.html
- [28] Rails. *railties CHANGELOG (8-0-stable)*. <https://github.com/rails/rails/blob/8-0-stable/railties/CHANGELOG.md>
- [29] AppSignal Blog. *Rails 8 Guide: Features, Requirements & Upgrade Path*. <https://blog.appsignal.com/2024/10/07/whats-new-in-ruby-on-rails-8.html>
- [30] Ruby on Rails. *Maintenance Policy*. <https://rubyonrails.org/maintenance>
- [31] HeroDevs. *Ruby on Rails End-of-Life Versions: The Dual Ruby + Rails EOL Problem*, 2026. <https://www.herodevs.com/blog-posts/ruby-on-rails-end-of-life-versions-the-dual-ruby-rails-eol-problem-enterprises-face-in-2026>
- [32] A. Piechowski. *Ruby 3.2 Is EOL: What You Actually Need to Do*, 2026. <https://piechowski.io/post/ruby-3-2-eol/>
- [33] Monterail. *The Evolution of Ruby on Rails*. <https://www.monterail.com/blog/what-is-ruby-on-rails-key-architectural-insights>
- [34] Hired. *Hired Releases “2023 State of Software Engineers Report”*. PR Newswire, 28 de febrero de 2023. <https://www.prnewswire.com/news-releases/hired-releases-2023-state-of-software-engineers-report-301757219.html>
- [35] Z. Zhan. *Analysis of Hired’s 2023 State of Software Engineers Report*. <https://agileway.substack.com/p/analysis-of-hireds-2023-state-of>
- [36] *Redis*. Wikipedia. <https://en.wikipedia.org/wiki/Redis>
- [37] *Introduction — Redis Documentation*. <https://redis-doc-test.readthedocs.io/en/latest/topics/introduction/>
- [38] Redis. *Scripting with Lua*. <https://redis.io/docs/latest/develop/programmability/eval-intro/>
- [39] Redis. *Redis is now available under the AGPLv3 open source license*, 2025. <https://redis.io/blog/agplv3/>
- [40] Valkey. *History*. <https://valkey.io/topics/history/>
- [41] *awesome scalability* (listado de lecturas de ingeniería). <https://github.com/binhnguyennus/awesome-scalability>
- [42] I. Fette y A. Melnikov. *RFC 6455: The WebSocket Protocol*. IETF, diciembre de 2011. <https://www.rfc-editor.org/rfc/rfc6455.html>
- [43] *WebSocket*. Wikipedia. <https://en.wikipedia.org/wiki/WebSocket>
- [44] WebSocket.org. *WebSocket Protocol: RFC 6455 Handshake, Frames & More*. <https://websocket.org/guides/websocket-protocol/>
- [45] noze. *WebSocket RFC 6455: the protocol for real-time web*. <https://www.noze.it/en/insights/websocket-rfc-6455/>
- [46] Slack. *Comparing HTTP & Socket Mode*. <https://docs.slack.dev/apis/events-api/comparing-http-socket-mode/>
- [47] Discord. *Gateway — Documentation*. <https://docs.discord.com/developers/events/gateway>
- [48] *Docker (software)*. Wikipedia. [https://en.wikipedia.org/wiki/Docker_\(software\)](https://en.wikipedia.org/wiki/Docker_(software))
- [49] *Docker (software)*. Wikipedia en español. [https://es.wikipedia.org/wiki/Docker_\(software\)](https://es.wikipedia.org/wiki/Docker_(software))
- [50] Docker. *Docker: Nine Years YOUNG*. <https://www.docker.com/blog/docker-nine-years-young/>
- [51] Data Center Knowledge. *Docker at Four: State of the Ecosystem, from 2013 to Today*. <https://www.datacenterknowledge.com/undefined-category/docker-at-four-state-of-the-ecosystem-from-2013-to-today>

- [52] Stack Overflow. *2024 Developer Survey: Technology*. <https://survey.stackoverflow.co/2024/technology/>